

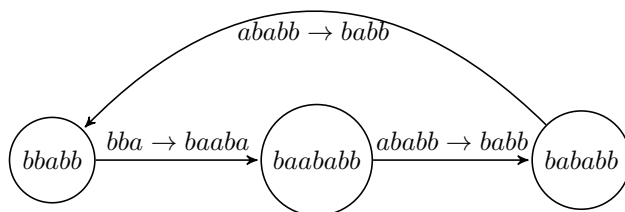
Лабораторная 1

Исходная система переписывания строк:

$aaaa \rightarrow a$
 $aaab \rightarrow b$
 $bbba \rightarrow ba$
 $bbbb \rightarrow bb$
 $ababb \rightarrow babb$
 $bba \rightarrow baaba$
 $bbb \rightarrow baabb$
 $baaaa \rightarrow baab$
 $bbaab \rightarrow baa$
 $bbabb \rightarrow abab$
 $baba \rightarrow baa$
 $babbaa \rightarrow babba$
 $babbab \rightarrow abb$

Завершимость

Рассмотрим слово $bbabb$:



Получили цикл, следовательно система незавершима. Сделаем систему завершимой, используя length-lexicographic order (сначала сравниваем по длине, если длины равны - лексикографическое сравнение). Поменяем в 2 правилах местами левую и правую части:

$baaba \rightarrow bba$
 $baabb \rightarrow bbb$

Получили завершимую систему с выбранным фундированным порядком.

Конфлюэнтность и пополняемость

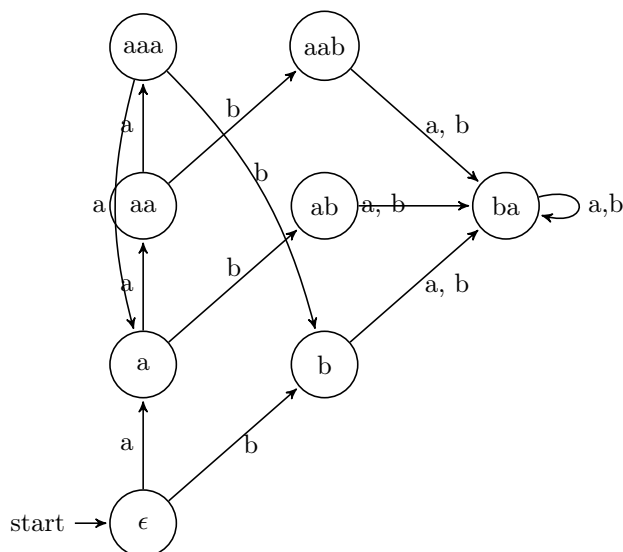
Рассмотрим конфлюэнтность. Найдем критическую пару между правилами $aaab \rightarrow b$ и $babbaa \rightarrow babba$, к примеру. Перекрытие - $babbaaab$, критическая пара - $(babbb, babbaab)$, первый элемент уже в нормальной форме, приведем тоже к нормальной форме - abb , применив правила $babbaa \rightarrow babba$ и затем $babbab \rightarrow abb$. Отсюда уже делаем вывод, что исходная система не является локально конфлюэнтной. Можно к примеру показать, что есть правила ведущие к одной нормальной форме: рассмотрим правила $aaaa \rightarrow a$ и $aaaaa \rightarrow a$. Их всевозможные перекрытия - это $aaaa$, $aaaaa$, $aaaaaa$, $aaaaaaa$ все они будут приведены при любой цепочки в a , aa , aaa , a соответственно, момент с критическими парами опустим ввиду очевидности. Ускоорим долгую и рутинную работу, написав программу, которая делает то же самое, что и я руками: между правилами находит перекрытие, приводит их критической паре, затем приводим каждый элемент к нормальной форме (она ищется единственным образом у меня, что не совсем корректно, но если у меня перестанут появляться в таком случае правила я могу говорить о конфлюэнтности системы, так как все критические пары в таком случае будут сходится к одной нормальной форме). После запуска алгоритма поиска критических пар и правил, которые они порождают получаем пополненную систему:

$aaaa \rightarrow a$
 $aaab \rightarrow b$
 $bbba \rightarrow ba$
 $bbbb \rightarrow bb$
 $ababb \rightarrow babb$
 $baaba \rightarrow bba$
 $baabb \rightarrow bbb$
 $bbaaa \rightarrow baab$
 $bbaab \rightarrow baa$
 $bbabb \rightarrow abab$
 $baba \rightarrow baa$
 $babbaa \rightarrow babba$
 $babbab \rightarrow abb$
 $babbb \rightarrow bab$
 $babbb \rightarrow abb$
 $babbb \rightarrow baaa$
 $babb \rightarrow bab$
 $bab \rightarrow abb$
 $baaa \rightarrow bab$
 $baab \rightarrow abb$
 $abb \rightarrow bb$
 $abaa \rightarrow baa$
 $baa \rightarrow bb$
 $bbaa \rightarrow bb$
 $bbaa \rightarrow baa$
 $bba \rightarrow aba$
 $baa \rightarrow ba$
 $bba \rightarrow baa$
 $bbb \rightarrow bb$
 $aba \rightarrow ba$
 $bb \rightarrow ba$

В этой системе все критические пары сходятся к общей нормальной форме, поэтому она конфлюэнтна. Получаем, что исходная система была пополняемой по Кнуту-Бендиксу.

Конечность

Из новых правил получаем автомат, состояния которого - нормальные формы системы:



Получаем 8 состояний автомата и столько же нормальных форм: ϵ , a , b , aa , ab , ba , aaa , aab .

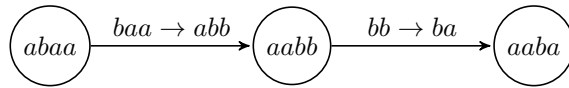
Ввиду конечного числа нормальных форм и предъявления автомата, система конечна.

Нахождение минимальной эквивалентной системы

Для начала заметим, что у нас в правилах есть такие правила, что они оба ведут в какую-то одну нормальную форму, например: $bbaab \rightarrow baa$ и $baba \rightarrow baa$, но при том может случиться так, что между левыми частями нет прохода, нужно чтобы во всех левых частях было не более одной нормальной формы. Тогда нужно их аккуратно упорядочить. Рассмотрим это на примере: $a \rightarrow c$, $b \rightarrow c$, a и b находятся в одном классе, но мы не сможем пройти от одного к другому, потому сделаем так $a \rightarrow b$, $b \rightarrow c$, это никак не повлияет на классы эквивалентности в системе, но гарантирует, что из слова можно прийти к любому меньшему (согласно порядка) слову. То есть можно делать так: $bbaab \rightarrow baba$ и $baba \rightarrow baa$. Такое действие допустимо ввиду конfluenceности, оно сохранит все классы эквивалентности. Также где-то уйдут избыточные правила. Прделав такую процедуру получаем:

$aaaa \rightarrow a$
 $aaab \rightarrow b$
 $bbba \rightarrow bbaa$
 $bbbb \rightarrow bbaa$
 $ababb \rightarrow aaabb$
 $baaba \rightarrow ababb$
 $baabb \rightarrow baaba$
 $bbaaa \rightarrow babbb$
 $bbaab \rightarrow babba$
 $bbabb \rightarrow abab$
 $baba \rightarrow baab$
 $babbaa \rightarrow bbaab$
 $babbab \rightarrow babbba$
 $babbb \rightarrow baabb$
 $babb \rightarrow baba$
 $bab \rightarrow baa$
 $baaa \rightarrow abaa$
 $baab \rightarrow baaa$
 $abb \rightarrow aba$
 $abaa \rightarrow bbb$
 $baa \rightarrow abb$
 $bbaa \rightarrow babbb$
 $bba \rightarrow bab$
 $bbb \rightarrow bba$
 $aba \rightarrow bb$
 $bb \rightarrow ba$

Теперь приступим к минимизации. Будем доказывать эвристически - делать левую часть как можно меньше посредством других правил, если так вышло, что левая часть при таком способе совпала с правой, то вычеркиваем такое правило из системы. К примеру: есть правило $abb \rightarrow aba$, применяем $bb \rightarrow ba$, получаем $aba \rightarrow aba$, которое вычеркиваем из нашей системы из-за ненужности. Или еще пример: есть правило $abaa \rightarrow bbb$, преобразуем левую часть:



Получаем правило $aaba \rightarrow bbb$. Прделав такую процедуру, мы сократим число правил и сделаем максимально минимальной левую часть правил. Получаем:

$$aaaa \rightarrow a$$

$$aaab \rightarrow b$$

$$bab \rightarrow baa$$

$$baa \rightarrow abb$$

$$aba \rightarrow bb$$

$$bb \rightarrow ba$$

Еще раз отметим, что левую часть правил невозможно сделать меньше, поэтому получившаяся система минимальна. Также заметим, что она также сохраняет те же классы эквивалентности, что и только что пополненная, потому она эквивалентна исходной.

Фазз-тестирование

Теперь алгоритм таков: 1. Нормализуем исходное слово, 2. Приводим переписанное слово в его нормальную форму (если привел не в ту-то ошибку), параллельно запоминая приведенные правила. 3. По этим правилам мы восстанавливаем путь уже от нормальной формы к переписанному слову, но уже перевернув правила. Таким образом, проход всегда идет в одном направлении-от начального слова к переписанному. И тестирование проходит быстрее.

Мета-тестирование

В начале заметим, что в автомате есть состояние-ловушка ba , да еще и такая, что при попадании в подграф, образованный этим состоянием и его соседями, мы останемся в этом подграфе. Это говорит по факту о том, что неизменным будет наличие буквы b : если её нет, то она и не появится, а если есть, то и в любом редуцированном слове она останется. Того первый инвариант - наличие буквы b в строке.

Далее вспомним, что у нас система минимальная конфлюэнтная. А значит я могу разбить язык на непересекающиеся классы, это классы как раз по нормальной форме. Каждое состояние образует некоторое такое множество. Тогда построим для каждого такого состояния язык, сделаем это посредством введения регулярного выражения, то есть мы перейдем от автомата к регулярному выражению. Таких регулярных выражений будет 8 - по числу состояний. Тогда инвариантной характеристикой будет удовлетворение тому же регулярному выражению, что и до применения правил. Тогда по стандартным правилам преобразований мы получим 8 регулярных выражений:

$$\begin{aligned}
L_\epsilon &= \epsilon \\
L_a &= a(a^3)^* \\
L_{aa} &= a^2(a^3)^* \\
L_{aaa} &= a^3(a^3)^* \\
L_{aab} &= a^2(a^3)^*b \\
L_{ab} &= a(a^3)^*b \\
L_b &= (a^3)^*b \\
L_{ba} &= (a^2|a|\epsilon)(a^3)^*b(a|b)(a|b)^*
\end{aligned}$$

Отмечу, что эти регулярные выражения попарно не пересекаются, а их объединением является $(a|b)^*$. Эта характеристика наиболее точно проверит принадлежность слова его классу эквивалентности.

Теперь рассмотрим, как еще можно различать (или отождествлять слова) из языка. Введем понятие различающего префикса-префикс минимальной длины, такой, что позволит производить различие между принадлежностью слов языку. По такому префиксу мы сможем явно определить принадлежность к определенному языку. Приведу такой алгоритм поиска:

1. Пустое слово всегда принадлежит L_ϵ , иначе:
2. Если нет буквы b в слове, то смотрим на остаток от деления числа букв a на 3: если он равен 1 - принадлежит L_a , равен 2 - принадлежит L_{aa} , равен 0 - принадлежит L_{aaa} , иначе:
3. Если после первой буквы b еще есть хотя бы одна буква - принадлежит L_{ba} , иначе:
4. Опять смотрим остаток от деления числа букв a перед первой b в слове на 3: остаток 0 - L_b , 1 - L_{ab} , 2 - L_{aab} .

Тогда, чтобы отождествить слова (или наоборот) достаточно лишь: 1. проверить пустое ли оно (при том если да, то последующее можно не проверять), 2. проверить наличие буквы b в слове, 3. проверить, есть ли после первой b еще символы (если да, то дальше можно не смотреть дальше), 4. Остаток от деления на 3 числа букв a перед первой b . Префиксом я это назвал, так как в подавляющем большинстве случаев хоть одна b в слове да найдется, а смотрим мы лишь на символы до нее и на наличие одного после, понятно, что в большинстве случаев это охватит лишь только небольшую стартовую часть слова. И инвариантом здесь является, что через данное введенное понятие я могу доказать, что слова лежат в одних классах для всех слов, то есть этот алгоритм проверки на "эквивалентность слов" применим будет ко всем словам, в том числе и после любой цепочки редуцирований этот алгоритм будет выполняться. В коде это будет смотреться понятнее, так как это алгоритм. А так в целом это еще один способ смотреть на принадлежность слов одному классу, но вместо регулярных выражений уже используется такой вот алгоритмический метод.